ACTL3142 Tutorial

Week 8: Tree-Based Methods

Nick Guo

School of Risk and Actuarial Studies, UNSW Sydney

T1 2026

Recap

From Week 7 (Cross-Validation & Regularisation):

- **Cross-validation**: hold out data in rotation to estimate OOS error. k-fold CV averages over k splits for a more stable estimate.
- **Ridge** (L_2): shrinks coefficients toward zero.
- **Lasso** (L_1): useful for variable selection (shrinks coefficients to exactly zero).
- Both trade a small bias increase for a larger variance decrease.

New idea: all of these methods are still **linear**. What if we partition the predictor space into regions and make simple predictions within each?

Today: **Decision Trees** (a fundamentally different approach) and **Ensemble Methods** (bagging, random forests, boosting).

Decision Trees: The Idea

A decision tree splits the predictor space into **non-overlapping rectangular regions** $R_1, R_2, \ldots, R_J$ using a sequence of binary rules.

For regression: predict $\hat{y} = \bar{y}_{R_j}$ (the mean of training observations in that region).
For classification: predict the **most common class** in the region.

Why trees are appealing:

- Easy to interpret and visualise (mirrors human decision-making)
- Handle interactions and non-linearities naturally
- No assumptions about the functional form of $f(X)$

The catch

A single tree typically has **high variance** and can overfit badly. This motivates pruning and, later, ensemble methods.

Growing a Regression Tree: Recursive Binary Splitting

We need a **loss function** to decide where to split. For regression trees we use **MSE loss**, where each region predicts $\hat{y}_{R_j} = \bar{y}_{R_j}$.

Finding the globally optimal partition is infeasible, so we use a **greedy, top-down** approach:

1. Consider all predictors X_j and all cutpoints s
2. Pick the (j, s) that minimises the combined MSE across the two child nodes:

$$\sum_{i:\, \mathbf{x}_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i:\, \mathbf{x}_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2$$

   where $R_1(j, s) = \{X \mid X_j < s\}$ and $R_2(j, s) = \{X \mid X_j \geq s\}$
3. Repeat on each resulting region until a stopping criterion is met

"Greedy" means each split is locally optimal. We never look ahead to see if a different split would produce a better tree overall. MSE is the standard loss here, but others (e.g. MAE) could be used.

Classification Trees: Loss Functions

For classification, MSE doesn't apply. We need a **loss function** that measures node impurity, and we **minimise** it at each split. Let $\hat{p}_{mk}$ = proportion of class k in node m:

Loss function	**Formula**
Classification error	$1 - \max_k(\hat{p}_{mk})$
Gini index	$\sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$
Entropy	$-\sum_{k=1}^{K} \hat{p}_{mk} \log(\hat{p}_{mk})$

All three are zero when a node is pure and maximised when classes are evenly mixed. At each split, pick the predictor and cutpoint giving the largest **reduction in loss**.

Which loss for splitting?

Gini and entropy are preferred for **growing** the tree: they are more

Cost-Complexity Pruning

A fully grown tree overfits. Rather than stopping early, we **grow a full tree first, then prune it back** (a weak split early on might lead to a great split below it).

Add a **penalty for tree size** to the loss. For regression (MSE loss):

$$C_\alpha(T) = \sum_{m=1}^{|T|} \sum_{i:\, \mathbf{x}_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha\, |T|$$

where $|T|$ is the number of leaves and $\alpha \geq 0$ controls the tradeoff. $\alpha = 0$ keeps the full tree, large α forces a smaller tree.

Procedure: grow a large tree T_0, then for each α find the best subtree minimising $C_\alpha(T)$. Choose α by **cross-validation**.

Same idea as λ in ridge/lasso: penalise complexity, tune the penalty by CV. For classification, replace MSE with Gini or entropy.

Why Ensemble Methods?

A single decision tree is:

- $+$ Interpretable and intuitive
- $-$ High variance: small changes in data $\rightarrow$ very different tree
- $-$ Often poor predictive accuracy compared to other methods

Key insight: averaging many high-variance, low-bias models **reduces variance** while keeping bias low.

If $Z_1, \ldots, Z_B$ are i.i.d. with variance σ^2, then $\text{Var}(\bar{Z}) = \sigma^2/B$.

Three ensemble approaches:

- **Bagging**: grow many deep trees on bootstrap samples, average them (variance reduction)
- **Random Forests**: bagging + randomise predictors at each split to decorrelate trees (better variance reduction)
- **Boosting**: grow stumps sequentially, each correcting previous errors (bias reduction)

Bagging (Bootstrap Aggregation)

Idea: we can't collect many independent training sets, but we *can* simulate them via **bootstrapping**.

Procedure:

1. Draw B bootstrap samples (sample n observations with replacement)
2. Fit a **deep, unpruned** tree to each bootstrap sample
3. Aggregate predictions:
   - Regression: $\hat{f}_{\text{bag}}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{*b}(\mathbf{x})$ (average)
   - Classification: majority vote across the B trees

Out-of-Bag (OOB) error: each bootstrap sample leaves out $\sim 1/3$ of observations. Predict each observation using only the trees that *didn't* train on it. This gives a free validation estimate without needing CV or a separate test set.

Increasing B does not cause overfitting. More trees just reduce variance further (with diminishing returns).

Random Forests

Problem with bagging: if one predictor is very strong, *every* bootstrapped tree splits on it first. The trees end up highly **correlated**, and averaging correlated quantities doesn't reduce variance effectively.

Random forests fix this: at each split, only consider a **random subset** of m predictors (out of p total).

- Typical choice: $m \approx \sqrt{p}$ (classification), $m \approx p/3$ (regression)
- Forces trees to explore different predictors $\rightarrow$ **decorrelates** them
- When $m = p$, random forests $=$ bagging

Why decorrelation helps

If trees have pairwise correlation ρ, the variance of their average is $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. Reducing ρ directly lowers the first term, which does **not** shrink with B.

Boosting: The Idea

Bagging/RF use deep trees (low bias, high variance) and reduce variance by averaging. **Boosting** takes the opposite approach: use **weak learners** (high bias, low variance) and **reduce bias** by combining them sequentially.

Algorithm (regression):

1. Set $\hat{f}(\mathbf{x}) = 0$ and residuals $r_i = y_i$
2. For $b = 1, \ldots, B$:
   - Fit a **stump** (or small tree) $\hat{f}^b$ with d splits to the **residuals**
   - Update: $\hat{f}(\mathbf{x}) \leftarrow \hat{f}(\mathbf{x}) + \lambda \hat{f}^b(\mathbf{x})$
   - Update residuals: $r_i \leftarrow r_i - \lambda \hat{f}^b(\mathbf{x}_i)$
3. Output: $\hat{f}(\mathbf{x}) = \sum_{b=1}^{B} \lambda \hat{f}^b(\mathbf{x})$

Each tree is intentionally simple (high bias on its own). By always fitting to the residuals, each new tree targets exactly what the ensemble still gets wrong.

Boosting: What do Iterations Look Like?

- **Tree 1**: residuals $= y_i$ (the full response). Fit a stump, get a rough first approximation. Update the ensemble by adding $\lambda \cdot \hat{f}^1(\mathbf{x})$.
- **Tree 2**: residuals $= y_i - \lambda \hat{f}^1(\mathbf{x}_i)$. The stump now targets whatever Tree 1 missed. Add $\lambda \cdot \hat{f}^2(\mathbf{x})$.
- **Tree 3**: residuals shrink further. Each tree chips away at the remaining error.

The learning rate λ controls how much of each tree we keep. Small λ (e.g. 0.01) means each tree contributes less, so we need more trees, but the model generalises better.

Three tuning parameters:

- B = number of trees. **Can overfit** if too large (unlike bagging/RF). Choose via CV.
- λ = learning rate (shrinkage). Smaller = slower learning but better performance.
- d = interaction depth. $d = 1$ (stumps) is standard. Controls interaction order.

Bagging vs Random Forests vs Boosting

	Bagging	**Random Forest**	**Boosting**
Trees built	Independently	Independently	Sequentially
Individual trees	Deep (low bias)	Deep (low bias)	Stumps (high bias)
Improves by	Reducing variance	Reducing variance	Reducing bias
Predictors/split	All p	Random $m < p$	All p
B can overfit?	No	No	**Yes**

Variable importance: we lose single-tree interpretability, but we can rank predictors by their contribution. For each predictor, sum the **reduction in loss** (MSE for regression, Gini for classification) across every split that uses it, averaged over all B trees. Predictors that produce large loss reductions are most important.

Boosting often gives the best predictive performance but requires more careful tuning than bagging or RF.

Q1: Gini Index, Classification Error, and Entropy

⋆ Solution (ISLR2, Q8.3) Consider the Gini index, classification error, and entropy in a simple classification setting with two classes. Create a single plot that displays each of these quantities as a function of $\hat{p}_{m1}$. The x-axis should display $\hat{p}_{m1}$, ranging from 0 to 1, and the y-axis should display the value of the Gini index, classification error, and entropy.

1. **Hint**: In a setting with two classes, $\hat{p}_{m1} = 1 - \hat{p}_{m2}$. You could make this plot by hand, but it will be much easier to make in `R`.

Q1: Walkthrough

For two classes ($K = 2$), let $p = \hat{p}_{m1}$ so $\hat{p}_{m2} = 1 - p$:

- **Classification error:** $E = 1 - \max(p, 1 - p)$
  - Piecewise linear (inverted V), peaks at $p = 0.5$ with value 0.5
- **Gini index:** $G = 2p(1 - p)$
  - Parabola, peaks at $p = 0.5$ with value 0.5
- **Entropy:** $D = -p \log(p) - (1 - p) \log(1 - p)$
  - Concave curve, peaks at $p = 0.5$

All three are zero at $p = 0$ and $p = 1$ (pure node) and maximised at $p = 0.5$ (maximum impurity). Gini and entropy curve more steeply near the extremes than classification error, making them more sensitive to purity changes when growing the tree.

Q2: Tree and Partition Correspondence

2. ⋆ Solution (ISLR2, Q8.4) This question relates to the plots in the textbook Figure 8.14, reproduced here as Figure 1:

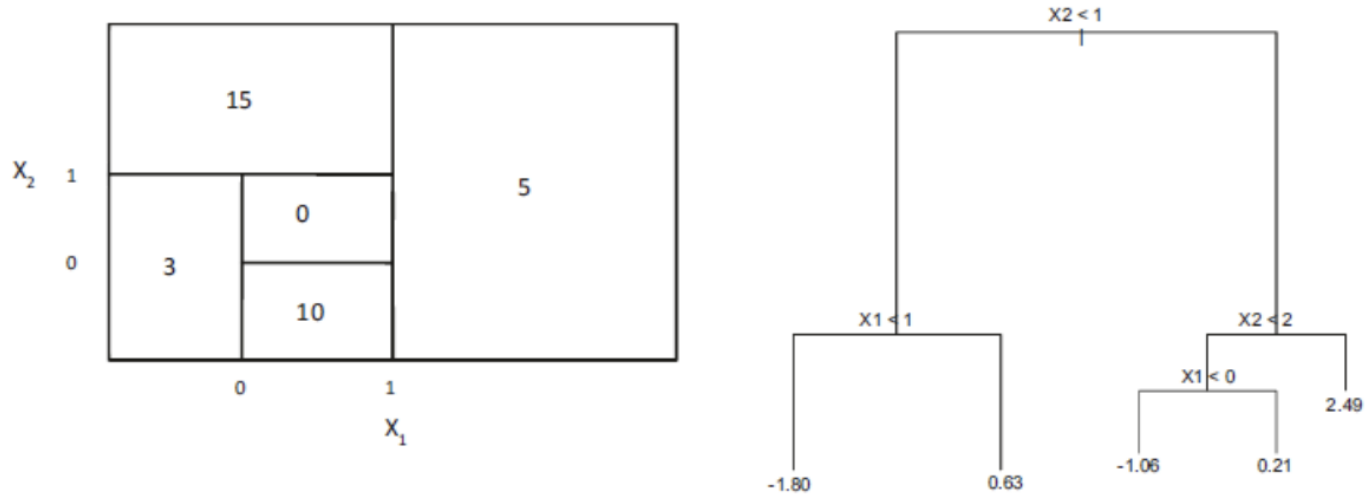

Figure 1: Left: A partition of the predictor space corresponding to Exercise 4a. Right: A tree corresponding to Exercise 4b.

a. Sketch the tree corresponding to the partition of the predictor space illustrated in the left-hand panel of Figure 8.14. The numbers inside the boxes indicate the mean of Y within each region.

b. Create a diagram similar to the left-hand panel of Figure 8.14, using the tree illustrated in the right-hand panel of the same figure. You should divide up the predictor space into the correct regions, and indicate the mean for each region.

Q2: Walkthrough

(a) Sketch the tree from the partition (left panel):

Read each boundary line as a split. Working top-down:

1. Root: $X_1 < 1$? [No $\rightarrow \hat{y} = 5$]
2. Left: $X_2 < 1$? [No $\rightarrow \hat{y} = 15$]
3. Left-left: $X_1 < 0$? [Yes $\rightarrow \hat{y} = 3$]
4. Remaining: $X_2 < 0$? [Yes $\rightarrow \hat{y} = 10$, No $\rightarrow \hat{y} = 0$]

(b) Partition from the tree (right panel):

Trace each root-to-leaf path to define rectangular regions:

- $X_2 < 1,\ X_1 < 1 \rightarrow \hat{y} = -1.80$
- $X_2 < 1,\ X_1 \geq 1 \rightarrow \hat{y} = 0.63$
- $1 \leq X_2 < 2,\ X_1 < 0 \rightarrow \hat{y} = -1.06$
- $1 \leq X_2 < 2,\ X_1 \geq 0 \rightarrow \hat{y} = 0.21$
- $X_2 \geq 2 \rightarrow \hat{y} = 2.49$

Each leaf in the tree corresponds to exactly one rectangular region in the partition. Draw horizontal/vertical lines at each split value.

Q3: Majority Vote vs Average Probability

3. ⋆ Solution (ISLR2, Q8.5) Suppose we produce ten bootstrapped samples from a data set containing red and green classes. We then apply a classification tree to each bootstrapped sample and, for a specific value of X, produce 10 estimates of $\mathbb{P}(\text{Class is Red}|X)$:

$$0.1, 0.15, 0.2, 0.2, 0.55, 0.6, 0.6, 0.65, 0.7, \text{ and } 0.75.$$

There are two common ways to combine these results together into a single class prediction. One is the majority vote approach discussed in this chapter. The second approach is to classify based on the average probability. In this example, what is the final classification under each of these two approaches?

Q3: Walkthrough

Given $P(\text{Red} \mid X)$ estimates from 10 bootstrap trees:
0.1, 0.15, 0.2, 0.2, 0.55, 0.6, 0.6, 0.65, 0.7, 0.75

Majority vote: each tree votes Red if $P > 0.5$, Green otherwise.

- Red votes: 6 trees $(0.55, 0.6, 0.6, 0.65, 0.7, 0.75)$
- Green votes: 4 trees $(0.1, 0.15, 0.2, 0.2)$
- Final classification: **Red** (6 vs 4)

Average probability:

$$\bar{P} = \frac{1}{10}(0.1+0.15+0.2+0.2+0.55+0.6+0.6+0.65+0.7+0.75) = 0.45$$

Since $0.45 < 0.5$, classify as **Green**.

Why do they disagree?

Majority vote ignores *how confident* each tree is. The 4 Green-voting trees are very confident (low P), while several Red-voting trees are barely past 0.5. Averaging preserves this information.

Applied Q1: Regression Trees on Carseats Data

Pull this question up on your side. Key pointers for each part:

(a) Split into train/test. Fit a regression tree to predict `Sales`. Use `tree()` from the `tree` package. Plot with `plot()` and `text()`. Report test MSE.

(b) Use `cv.tree()` to find the optimal tree size. Does pruning improve test MSE? Plot CV deviance vs tree size.

(c) Bagging: use `randomForest(..., mtry = p)` where p is the number of predictors. Report test MSE and variable importance via `importance()`.

(d) Random forest: use `randomForest()` with default `mtry`. Try different values of `mtry` and see how test MSE changes. Which variables are most important?

Compare MSEs across all methods. Ensemble methods should substantially outperform a single tree.

Applied Q3: Boosting on Hitters Data

Pull this question up on your side. Key pointers for each part:

(a) Remove rows with missing `Salary`. Log-transform `Salary`. Use first 200 observations as training, rest as test.

(b) Fit boosted trees using `gbm()` with a grid of λ values (e.g. 10^{-3} to 1). Plot training MSE vs λ.

(c) Plot test MSE vs λ. Find the optimal shrinkage parameter.

(d) Compare test MSE with OLS (`lm()`) and lasso (`cv.glmnet()`).

(e) Use `summary()` on the boosted model to get variable importance. Which predictors matter most?

Key takeaway: boosting with a small λ and many trees often beats linear models when the true relationship is non-linear.

Thanks for coming! See you next week.